

Redpwn 2020 Crypto - Itsy Bitsy

deuterium — 2020-06-27

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/redpwn/crypto/itsy-bitsy/2020-06-27-redpwn-2020-Itsy-Bitsy.html>

Abstract

CTF writeup: Leading bits in this random XOR keystream insist on being 1. Their predictable habits expose the plaintext in redpwnCTF 2020's Itsy Bitsy.

itsy-bitsy

Description

The itsy-bitsy spider climbed up the water spout...

```
nc 2020.redpwnc.tf 31284
```

Files

- [itsy-bitsy.py](#)

Lets quickly take a look at now the connection works

```
nc 2020.redpwnc.tf 31284
Enter an integer i such that i > 0: 1
Enter an integer j such that j > i > 0: 2
Ciphertext:
00100010010111011110000011000000100000110001111001001111000110001000010010011001111100
11100100000001101101010101001100001100010010000001111000000010100001001100100001000000
0101010001010101010110100110111001101000001000111000010100001101100100101011001010100
0000100001011110000100010100010010110100000
```

Let us quickly go through the required functions of the source

```
#!/usr/bin/env python3

from Crypto.Random.random import randint

def str_to_bits(s):
    bit_str = ''
    for c in s:
        i = ord(c)
        bit_str += bin(i)[2:]
    return bit_str

def recv_input():
    i = input('Enter an integer i such that i > 0: ')
    j = input('Enter an integer j such that j > i > 0: ')
```

```

try:
    i = int(i)
    j = int(j)
    if i <= 0 or j <= i:
        raise Exception
except:
    print('Error! You must adhere to the restrictions!')
    exit()
return i,j

def generate_random_bits(lower_bound, upper_bound, number_of_bits):
    bit_str = ''
    while len(bit_str) < number_of_bits:
        r = randint(lower_bound, upper_bound)
        bit_str += bin(r)[2:]
    return bit_str[:number_of_bits]

def bit_str_xor(bit_str_1, bit_str_2):
    xor_res = ''
    for i in range(len(bit_str_1)):
        bit_1 = bit_str_1[i]
        bit_2 = bit_str_2[i]
        xor_res += str(int(bit_1) ^ int(bit_2))
    return xor_res

def main():
    with open('flag.txt','r') as f:
        flag = f.read()
    for c in flag:
        i = ord(c)
        assert i in range(2**6,2**7)
    flag_bits = str_to_bits(flag)
    i,j = recv_input()
    lb = 2**i
    ub = 2**j - 1
    n = len(flag_bits)
    random_bits = generate_random_bits(lb,ub,n)
    encrypted_bits = bit_str_xor(flag_bits,random_bits)
    print(f'Ciphertext: {encrypted_bits}')

if __name__ == '__main__':
    main()

```

```

def generate_random_bits(lower_bound, upper_bound, number_of_bits):
    bit_str = ''
    while len(bit_str) < number_of_bits:
        r = randint(lower_bound, upper_bound)
        bit_str += bin(r)[2:]
    return bit_str[:number_of_bits]

```

It will generate a random bitstring of length `number_of_bits` from `lower_bound` to `upper_bound` and if the `lower_bound` itself is greater than the requirement, it will truncate it.

We are asked for input for `i` and `j` which are converted to lower and upper bounds as `2**i` to `2**j - 1`.

The generated bitstream is xored (bit-wise) with the flag and returned

Observation

If one sets i and j to i and $i+1$, the bounds are 2^{**i} and $2^{**i+1} - 1$, which are both i bits in length.

The peculiarity being the first bit would always be 1 because of 2^{**i} and all the bits at positions multiples of i would be 1

Example

```
generate_random_bits(2**1,2**(1+1)-1,80)
'1010111111101011101011111011101010101011111011111101010101011111111011111011111011'
```

Notice the 0th, 2nd, 4th index and so on is always 1 My unoptimized script `solve.py` which just concerns about knowing the bit at position i by sending the index $i-1$ and i .

```
from pwn import remote

HOST, PORT = "2020.redpwn.tf", 31284

def get_ct(num_bits):
    REM = remote(HOST, PORT)
    REM.sendline(str(num_bits - 1).encode())
    REM.sendline(str(num_bits).encode())
    data = REM.recvline()
    ciphertext = data.split(b':')[-1].strip()
    #print(data)
    REM.close()
    return ciphertext

flag_bits = bytearray(301) #flag size known
flag_bits[0] = 49 # ord('f') known from the first character 'f' of flag
flag_bits[1] = 49 # ord('l')
for num_bits in range(2,301):
    ct = get_ct(num_bits)
    flag_bits[num_bits] = 48 if ct[num_bits] == 49 else 49

print(bits_to_str(flag_bits))
```

This can be improved just by noting the fact we knew most of the bits i before reaching the index i , we will only make a request if bit i is not known

```
for num_bits in range(2,301):
    if flag_bits[num_bits] == 0:
        ct = get_ct(num_bits)
        for i in range(num_bits, 301, num_bits):
            flag_bits[i] = 48 if ct[i] == 49 else 49
print(bits_to_str(flag_bits))
```

Which takes 62 requests instead of 300 as in original

flag-bits'leaking'out'down'the'water'spout"

Interactive controls are omitted from this PDF. See the canonical HTML version.